

Haylee Paredes

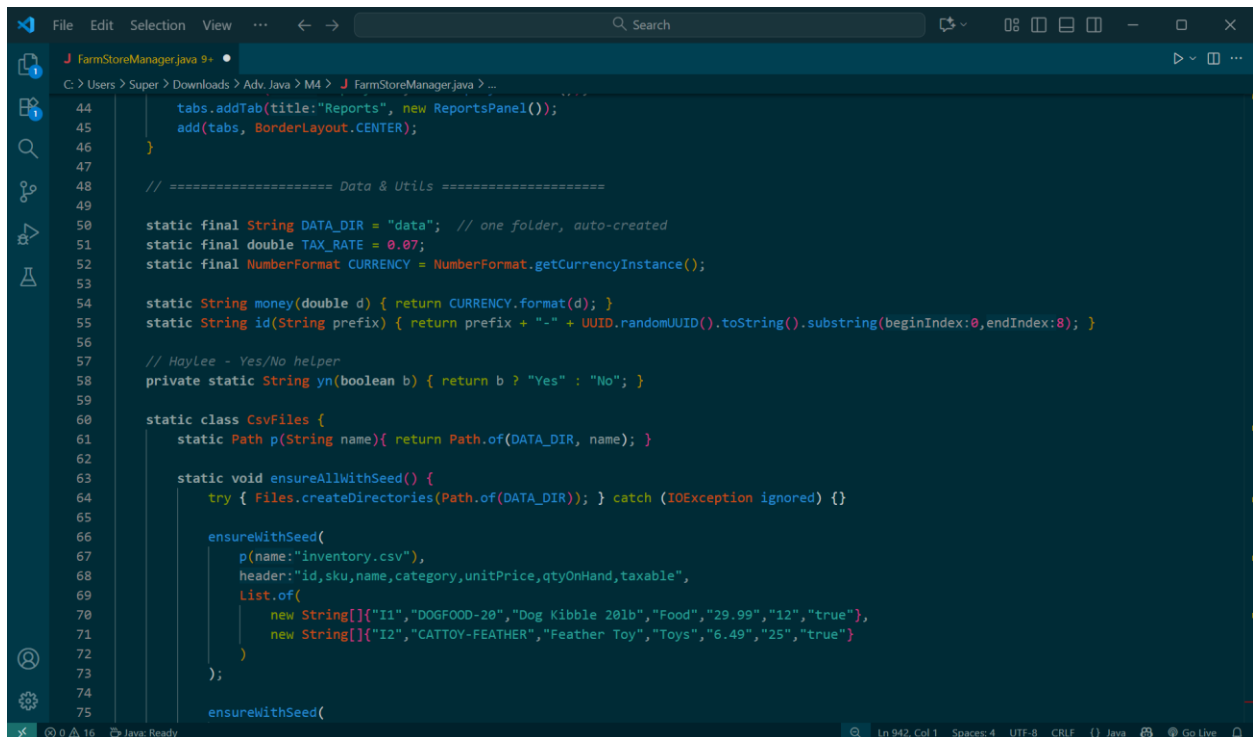
Oct. 30, 2025

CSC-251-0901

Advanced Java Programming

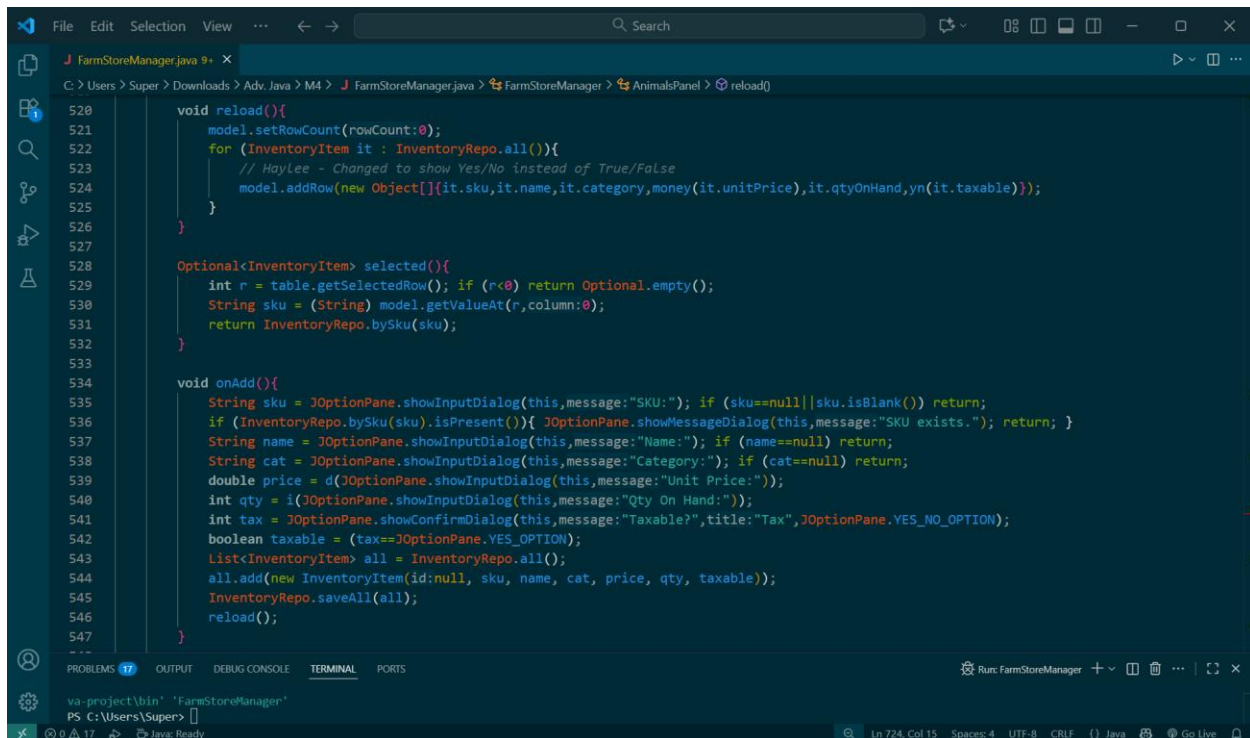
Lines 58, 524, and 725 are exchanging True/False for Yes/No.

Line 58:



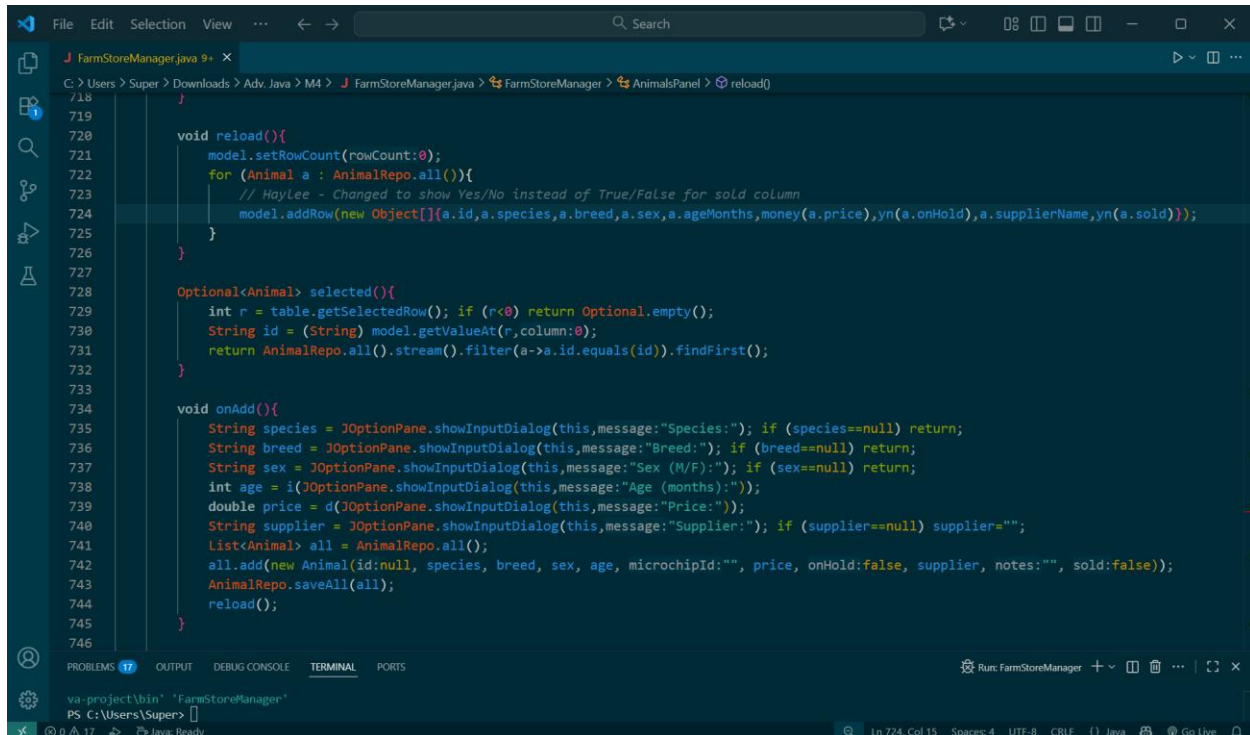
```
44     tabs.addTab(title:"Reports", new ReportsPanel());
45     add(tabs, BorderLayout.CENTER);
46 }
47
48 // ===== Data & Utils =====
49
50 static final String DATA_DIR = "data"; // one folder, auto-created
51 static final double TAX_RATE = 0.07;
52 static final NumberFormat CURRENCY = NumberFormat.getCurrencyInstance();
53
54 static String money(double d) { return CURRENCY.format(d); }
55 static String id(String prefix) { return prefix + "-" + UUID.randomUUID().toString().substring(beginIndex:0,endIndex:8); }
56
57 // Haylee - Yes/No helper
58 private static String yn(boolean b) { return b ? "Yes" : "No"; }
59
60 static class CsvFiles {
61     static Path p(String name){ return Path.of(DATA_DIR, name); }
62
63     static void ensureAllWithSeed() {
64         try { Files.createDirectories(Path.of(DATA_DIR)); } catch (IOException ignored) {}
65
66         ensureWithSeed(
67             p(name:"inventory.csv"),
68             header:"id,sku,name,category,unitPrice,qtyOnHand,taxable",
69             List.of(
70                 new String[]{"I1","DOGFOOD-20","Dog Kibble 20lb","Food","29.99","12","true"},
71                 new String[]{"I2","CATTOY-FEATHER","Feather Toy","Toys","6.49","25","true"}
72             )
73         );
74
75         ensureWithSeed(
```

Line 524:



```
520     void reload(){
521         model.setRowCount(rowCount:0);
522         for (InventoryItem it : InventoryRepo.all()){
523             // HayLee - Changed to show Yes/No instead of True/False
524             model.addRow(new Object[]{it.sku,it.name,it.category,money(it.unitPrice),it.qtyOnHand,yn(it.taxable)});
525         }
526     }
527
528     Optional<InventoryItem> selected(){
529         int r = table.getSelectedRow(); if (r<0) return Optional.empty();
530         String sku = (String) model.getValueAt(r,column:0);
531         return InventoryRepo.bySku(sku);
532     }
533
534     void onAdd(){
535         String sku = JOptionPane.showInputDialog(this,message:"SKU:"); if (sku==null||sku.isBlank()) return;
536         if (InventoryRepo.bySku(sku).isPresent()){ JOptionPane.showMessageDialog(this,message:"SKU exists."); return; }
537         String name = JOptionPane.showInputDialog(this,message:"Name:"); if (name==null) return;
538         String cat = JOptionPane.showInputDialog(this,message:"Category:"); if (cat==null) return;
539         double price = d(JOptionPane.showInputDialog(this,message:"Unit Price:"));
540         int qty = i(JOptionPane.showInputDialog(this,message:"Qty On Hand:"));
541         int tax = JOptionPane.showConfirmDialog(this,message:"Taxable?",title:"Tax",JOptionPane.YES_NO_OPTION);
542         boolean taxable = (tax==JOptionPane.YES_OPTION);
543         List<InventoryItem> all = InventoryRepo.all();
544         all.add(new InventoryItem(id:null, sku, name, cat, price, qty, taxable));
545         InventoryRepo.saveAll(all);
546         reload();
547     }
548 }
```

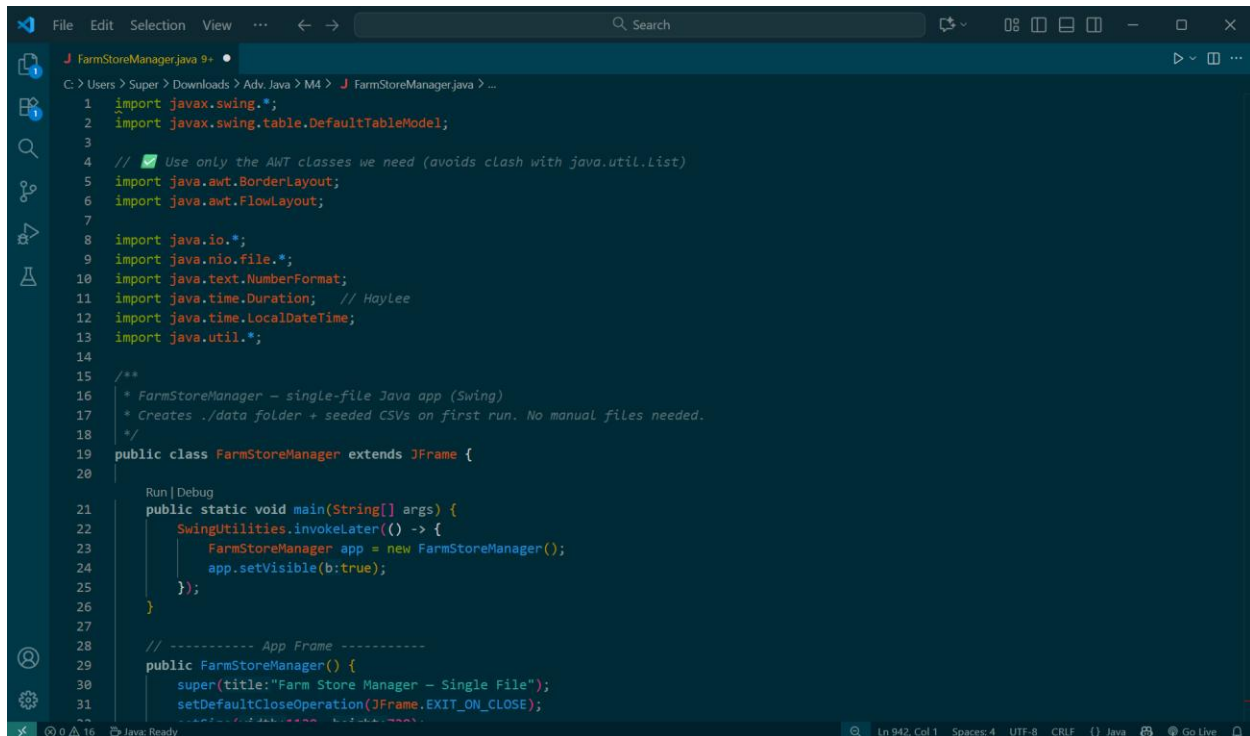
Line 724:



```
718     }
719
720     void reload(){
721         model.setRowCount(rowCount:0);
722         for (Animal a : AnimalRepo.all()){
723             // HayLee - Changed to show Yes/No instead of True/False for sold column
724             model.addRow(new Object[]{a.id,a.species,a.breed,a.sex,a.ageMonths,money(a.price),yn(a.onHold),a.supplierName,yn(a.sold)});
725         }
726     }
727
728     Optional<Animal> selected(){
729         int r = table.getSelectedRow(); if (r<0) return Optional.empty();
730         String id = (String) model.getValueAt(r,column:0);
731         return AnimalRepo.all().stream().filter(a->a.id.equals(id)).findFirst();
732     }
733
734     void onAdd(){
735         String species = JOptionPane.showInputDialog(this,message:"Species:"); if (species==null) return;
736         String breed = JOptionPane.showInputDialog(this,message:"Breed:"); if (breed==null) return;
737         String sex = JOptionPane.showInputDialog(this,message:"Sex (M/F):"); if (sex==null) return;
738         int age = i(JOptionPane.showInputDialog(this,message:"Age (months):"));
739         double price = d(JOptionPane.showInputDialog(this,message:"Price:"));
740         String supplier = JOptionPane.showInputDialog(this,message:"Supplier:"); if (supplier==null) supplier="";
741         List<Animal> all = AnimalRepo.all();
742         all.add(new Animal(id:null, species, breed, sex, age, microchipId:"", price, onHold:false, supplier, notes:"", sold:false));
743         AnimalRepo.saveAll(all);
744         reload();
745     }
746 }
```

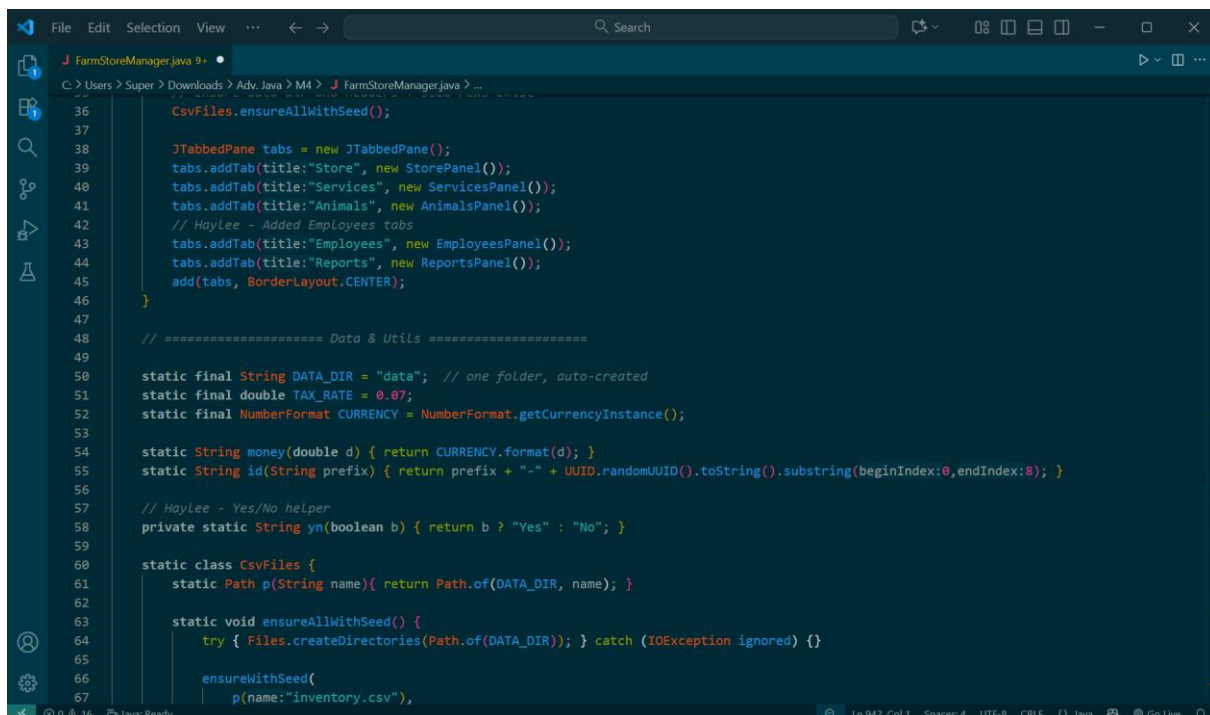
Lines 11, 43, 106-114, 249-272, 419-475, 784-910, and 934-937 are adding an employee's section to track clock ins and clock outs, and their paychecks.

Line 11:



```
1  import javax.swing.*;
2  import javax.swing.table.DefaultTableModel;
3
4  // Use only the AWT classes we need (avoids clash with java.util.List)
5  import java.awt.BorderLayout;
6  import java.awt.FlowLayout;
7
8  import java.io.*;
9  import java.nio.file.*;
10 import java.text.NumberFormat;
11 import java.time.Duration; // HayLee
12 import java.time.LocalDate;
13 import java.util.*;
14
15 /**
16  * FarmStoreManager - single-file Java app (Swing)
17  * Creates ./data folder + seeded CSVs on first run. No manual files needed.
18  */
19 public class FarmStoreManager extends JFrame {
20
21     Run | Debug
22     public static void main(String[] args) {
23         SwingUtilities.invokeLater(() -> {
24             FarmStoreManager app = new FarmStoreManager();
25             app.setVisible(b:true);
26         });
27
28         // ----- App Frame -----
29         public FarmStoreManager() {
30             super(title:"Farm Store Manager - Single File");
31             setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
32         }
33     }
34 }
```

Line 43:



```
36     CsvFiles.ensureAllWithSeed();
37
38     JTabbedPane tabs = new JTabbedPane();
39     tabs.addTab(title:"Store", new StorePanel());
40     tabs.addTab(title:"Services", new ServicesPanel());
41     tabs.addTab(title:"Animals", new AnimalsPanel());
42     // HayLee - Added Employees tabs
43     tabs.addTab(title:"Employees", new EmployeesPanel());
44     tabs.addTab(title:"Reports", new ReportsPanel());
45     add(tabs, BorderLayout.CENTER);
46 }
47
48 // ----- Data & Utils -----
49
50 static final String DATA_DIR = "data"; // one folder, auto-created
51 static final double TAX_RATE = 0.07;
52 static final NumberFormat CURRENCY = NumberFormat.getCurrencyInstance();
53
54 static String money(double d) { return CURRENCY.format(d); }
55 static String id(String prefix) { return prefix + "-" + UUID.randomUUID().toString().substring(beginIndex:0,endIndex:8); }
56
57 // HayLee - Yes/No helper
58 private static String yn(boolean b) { return b ? "Yes" : "No"; }
59
60 static class CsvFiles {
61     static Path p(String name){ return Path.of(DATA_DIR, name); }
62
63     static void ensureAllWithSeed() {
64         try { Files.createDirectories(Path.of(DATA_DIR)); } catch (IOException ignored) {}
65
66         ensureWithSeed(
67             p(name:"inventory.csv"),
68         );
69     }
70 }
```

Lines 106-114:

```
File Edit Selection View ... Search
J FarmStoreManager.java 9+
C:\Users\Super\Downloads\Adv.Java\M4> J FarmStoreManager.java ...
98         new String[]{"E1", "Wellness Check", "General check-up", "35.00", "30"}
99     };
100 }
101
102 ensureHeaderOnly(p(name:"appointments.csv"), header:"id,customerId,animalId,serviceId,start,end,status,paidAmount");
103 ensureHeaderOnly(p(name:"sales.csv"), header:"id,dateTime,customerId,subTotal,tax,total,paidCash,paidCard,linesJson");
104
105 // HayLee - Employees and TimeClock CSVs
106 ensureWithSeed(
107     p(name:"employees.csv"),
108     header:"id,name,hourlyRate,active",
109     List.of(
110         new String[]{"E1", "Alice Johnson", "15.50", "true"},
111         new String[]{"E2", "Marco Diaz", "14.00", "true"}
112     )
113 );
114 ensureHeaderOnly(p(name:"timeclock.csv"), header:"id,employeeId,clockIn,clockOut,hours,pay");
115 }
116
117 static void ensureWithSeed(Path path, String header, List<String[]> seedRows) {
118     if (!Files.exists(path) || isEmptyData(path)) {
119         try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(path))) {
120             pw.println(header);
121             for (String[] r : seedRows) pw.println(String.join(delimiter, ", ", safe(r)));
122         } catch (IOException e) { e.printStackTrace(); }
123     }
124 }
125
126 static void ensureHeaderOnly(Path path, String header) {
127     if (!Files.exists(path) || isEmptyData(path)) {
128         try (PrintWriter pw = new PrintWriter(Files.newBufferedWriter(path))) {
129             pw.println(header);
130         } catch (IOException e) { e.printStackTrace(); }
131     }
132 }
```

Lines 249-272:

```
File Edit Selection View ... Search
J FarmStoreManager.java 9+
C:\Users\Super\Downloads\Adv.Java\M4> J FarmStoreManager.java ... FarmStoreManager > Employee
247 // ===== Employee and TimeEntry Models =====
248 // HayLee - Employee and TimeEntry classes
249 static class Employee {
250     String id, name;
251     double hourlyRate;
252     boolean active;
253     Employee(String id, String name, double hourlyRate, boolean active) {
254         this.id = id == null ? id(prefix:"E") : id;
255         this.name = name;
256         this.hourlyRate = hourlyRate;
257         this.active = active;
258     }
259 }
260 static class TimeEntry {
261     String id, employeeId;
262     LocalDateTime clockIn, clockOut;
263     double hours, pay;
264     TimeEntry(String id, String employeeId, LocalDateTime in, LocalDateTime out, double hours, double pay) {
265         this.id = id == null ? id(prefix:"TC") : id;
266         this.employeeId = employeeId;
267         this.clockIn = in;
268         this.clockOut = out;
269         this.hours = hours;
270         this.pay = pay;
271     }
272 }
273
274 // ===== Storage (CSV-backed) =====
275
```

Lines 419-475:

```
File Edit Selection View ... Search
J FarmStoreManager.java 9+ x
C:\Users\Super\Downloads> Adv.Java> M4> J FarmStoreManager.java> FarmStoreManager> Employee
414 static String escape(String s){ if (s==null) return ""; return s.replace(target: '\\',replacement: '\\').replace(target: ';',replacement: '/');
415 }
416
417 // ===== Employee and TimeEntry Repos =====
418 // HayLee - EmployeeRepo and TimeRepo classes
419 static class EmployeeRepo {
420     static List<Employee> all() {
421         List<Employee> list = new ArrayList<>();
422         for (String[] r : CsvFiles.read(CsvFiles.p(name:"employees.csv"))){
423             if (r.length < 4) continue;
424             list.add(new Employee(r[0], r[1], d(r[2]), b(r[3])));
425         }
426         return list;
427     }
428     static void saveAll(List<Employee> items){
429         List<String[]> rows = new ArrayList<>();
430         for (Employee e : items){
431             rows.add(new String[]{e.id, e.name, Double.toString(e.hourlyRate), Boolean.toString(e.active)});
432         }
433         CsvFiles.write(CsvFiles.p(name:"employees.csv"), header:"id,name,hourlyRate,active", rows);
434     }
435     static Optional<Employee> byId(String id){
436         return all().stream().filter(e->e.id.equals(id)).findFirst();
437     }
438 }
439
440 static class TimeRepo {
441     static List<TimeEntry> all() {
```

```
File Edit Selection View ... Search
J FarmStoreManager.java 9+ x
C:\Users\Super\Downloads> Adv.Java> M4> J FarmStoreManager.java> FarmStoreManager> TimeRepo> all()
441 static List<TimeEntry> all() {
442     List<TimeEntry> list = new ArrayList<>();
443     for (String[] r : CsvFiles.read(CsvFiles.p(name:"timeclock.csv"))){
444         if (r.length < 6) continue;
445         LocalDateTime cin = r[2].isBlank() ? null : LocalDateTime.parse(r[2]);
446         LocalDateTime cout = r[3].isBlank() ? null : LocalDateTime.parse(r[3]);
447         list.add(new TimeEntry(r[0], r[1], cin, cout, d(r[4]), d(r[5])));
448     }
449     return list;
450 }
451 static void saveAll(List<TimeEntry> items){
452     List<String[]> rows = new ArrayList<>();
453     for (TimeEntry t : items){
454         rows.add(new String[]{
455             t.id, t.employeeId,
456             t.clockIn == null ? "" : t.clockIn.toString(),
457             t.clockOut == null ? "" : t.clockOut.toString(),
458             Double.toString(t.hours),
459             Double.toString(t.pay)
460         });
461     }
462     CsvFiles.write(CsvFiles.p(name:"timeclock.csv"), header:"id,employeeId,clockIn,clockOut,hours,pay", rows);
463 }
464 static Optional<TimeEntry> openShiftFor(String empId){
465     return all().stream().filter(t -> empId.equals(t.employeeId) && t.clockOut==null).findFirst();
466 }
467
468 static double sumHours(String empId){
```



```
File Edit Selection View ... Search
J FarmStoreManager.java 9+ x
C:\Users\Super\Downloads\Adv.Java\M4\J FarmStoreManager.java > FarmStoreManager > TimeRepo > sumHours(String)
468 static double sumHours(String empId){
469     return all().stream().filter(t -> empId.equals(t.employeeId)).mapToDouble(t -> t.hours).sum();
470 }
471
472 static double sumPay(String empId){
473     return all().stream().filter(t -> empId.equals(t.employeeId)).mapToDouble(t -> t.pay).sum();
474 }
475 }
476
477 // parsing helpers
478 static int i(String s){ try { return Integer.parseInt(s.trim()); } catch(Exception e){ return 0; } }
479 static double d(String s){ try { return Double.parseDouble(s.trim()); } catch(Exception e){ return 0.0; } }
480 static boolean b(String s){ return "true".equalsIgnoreCase(s.trim()); }
481 static String n(String s){ return (s==null || s.isBlank())? null : s; }
482 static String nn(String s){ return s==null? "" : s; }
483
484 // sales compute
485 static void computeTotals(Sale sale){
486     double sub=0, tax=0;
487     for (SaleLine l : sale.lines){ sub += l.lineTotal; if (l.taxable) tax += l.lineTotal*TAX_RATE; }
488     sale.subTotal=sub; sale.tax=tax; sale.total=sub+tax;
489 }
490
491 // ===== Panels =====
492
493 // ---- Store (Inventory & Item Sales) ----
494 class StorePanel extends JPanel {
495     DefaultTableModel model = new DefaultTableModel(new Object[]{"SKU","Name","Category","Price","Qty","Taxable"}, 0){
496         public boolean isCellEditable(int r,int c){return false;}
497     };
498     JTable table = new JTable(model);
499
500     StorePanel() {
501         setLayout(new BorderLayout());
502         add(new JScrollPane(table), BorderLayout.CENTER);
503         JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT));
504         JButton add = new JButton(text:"Add Employee");
505         JButton edit = new JButton(text:"Edit Employee");
506         JButton toggle = new JButton(text:"Toggle Active");
507         JButton clockIn = new JButton(text:"Clock In");
508         JButton clockOut = new JButton(text:"Clock Out");
509         actions.add(add); actions.add(edit); actions.add(toggle); actions.add(clockIn); actions.add(clockOut);
510         add(actions, BorderLayout.NORTH);
511
512         add.addActionListener(e -> onAdd());
513         edit.addActionListener(e -> onEdit());
514         toggle.addActionListener(e -> onToggleActive());
515         clockIn.addActionListener(e -> onClockIn());
516         clockOut.addActionListener(e -> onClockOut());
517
518         reload();
519     }
520 }
521
522 // ---- Employees and Time Clock ----
523 // HayLee - EmployeesPanel class
524 class EmployeesPanel extends JPanel {
525     DefaultTableModel model = new DefaultTableModel(new Object[]{"ID","Name","Hourly Rate","Active","Clock In"},0){
526         public boolean isCellEditable(int r,int c){return false;}
527     };
528     JTable table = new JTable(model);
529
530     EmployeesPanel() {
531         setLayout(new BorderLayout());
532         add(new JScrollPane(table), BorderLayout.CENTER);
533         JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT));
534         JButton add = new JButton(text:"Add Employee");
535         JButton edit = new JButton(text:"Edit Employee");
536         JButton toggle = new JButton(text:"Toggle Active");
537         JButton clockIn = new JButton(text:"Clock In");
538         JButton clockOut = new JButton(text:"Clock Out");
539         actions.add(add); actions.add(edit); actions.add(toggle); actions.add(clockIn); actions.add(clockOut);
540         add(actions, BorderLayout.NORTH);
541
542         add.addActionListener(e -> onAdd());
543         edit.addActionListener(e -> onEdit());
544         toggle.addActionListener(e -> onToggleActive());
545         clockIn.addActionListener(e -> onClockIn());
546         clockOut.addActionListener(e -> onClockOut());
547
548         reload();
549     }
550 }
551
552 // ---- Time Clock ----
553 // HayLee - TimeClockPanel class
554 class TimeClockPanel extends JPanel {
555     DefaultTableModel model = new DefaultTableModel(new Object[]{"ID","Name","Hourly Rate","Active","Clock In"},0){
556         public boolean isCellEditable(int r,int c){return false;}
557     };
558     JTable table = new JTable(model);
559
560     TimeClockPanel() {
561         setLayout(new BorderLayout());
562         add(new JScrollPane(table), BorderLayout.CENTER);
563         JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT));
564         JButton add = new JButton(text:"Add Employee");
565         JButton edit = new JButton(text:"Edit Employee");
566         JButton toggle = new JButton(text:"Toggle Active");
567         JButton clockIn = new JButton(text:"Clock In");
568         JButton clockOut = new JButton(text:"Clock Out");
569         actions.add(add); actions.add(edit); actions.add(toggle); actions.add(clockIn); actions.add(clockOut);
570         add(actions, BorderLayout.NORTH);
571
572         add.addActionListener(e -> onAdd());
573         edit.addActionListener(e -> onEdit());
574         toggle.addActionListener(e -> onToggleActive());
575         clockIn.addActionListener(e -> onClockIn());
576         clockOut.addActionListener(e -> onClockOut());
577
578         reload();
579     }
580 }
581
582 // ---- Main Method ----
583 // HayLee - Main Method
584 public static void main(String[] args) {
585     JFrame frame = new JFrame("FarmStoreManager");
586     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
587     frame.add(new StorePanel());
588     frame.add(new EmployeesPanel());
589     frame.add(new TimeClockPanel());
590     frame.pack();
591     frame.setVisible(true);
592 }
593
594 // ===== End of File =====
```

Lines 784-910:

```
File Edit Selection View ... Search
J FarmStoreManager.java 9+ x
C:\Users\Super\Downloads\Adv.Java\M4\J FarmStoreManager.java > FarmStoreManager > EmployeesPanel
780 }
781
782 // ---- Employees and Time Clock ----
783 // HayLee - EmployeesPanel class
784 class EmployeesPanel extends JPanel {
785     DefaultTableModel model = new DefaultTableModel(new Object[]{"ID","Name","Hourly Rate","Active","Clock In"},0){
786         public boolean isCellEditable(int r,int c){return false;}
787     };
788     JTable table = new JTable(model);
789
790     EmployeesPanel() {
791         setLayout(new BorderLayout());
792         add(new JScrollPane(table), BorderLayout.CENTER);
793         JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT));
794         JButton add = new JButton(text:"Add Employee");
795         JButton edit = new JButton(text:"Edit Employee");
796         JButton toggle = new JButton(text:"Toggle Active");
797         JButton clockIn = new JButton(text:"Clock In");
798         JButton clockOut = new JButton(text:"Clock Out");
799         actions.add(add); actions.add(edit); actions.add(toggle); actions.add(clockIn); actions.add(clockOut);
800         add(actions, BorderLayout.NORTH);
801
802         add.addActionListener(e -> onAdd());
803         edit.addActionListener(e -> onEdit());
804         toggle.addActionListener(e -> onToggleActive());
805         clockIn.addActionListener(e -> onClockIn());
806         clockOut.addActionListener(e -> onClockOut());
807
808         reload();
809     }
810 }
811
812 // ---- Time Clock ----
813 // HayLee - TimeClockPanel class
814 class TimeClockPanel extends JPanel {
815     DefaultTableModel model = new DefaultTableModel(new Object[]{"ID","Name","Hourly Rate","Active","Clock In"},0){
816         public boolean isCellEditable(int r,int c){return false;}
817     };
818     JTable table = new JTable(model);
819
820     TimeClockPanel() {
821         setLayout(new BorderLayout());
822         add(new JScrollPane(table), BorderLayout.CENTER);
823         JPanel actions = new JPanel(new FlowLayout(FlowLayout.LEFT));
824         JButton add = new JButton(text:"Add Employee");
825         JButton edit = new JButton(text:"Edit Employee");
826         JButton toggle = new JButton(text:"Toggle Active");
827         JButton clockIn = new JButton(text:"Clock In");
828         JButton clockOut = new JButton(text:"Clock Out");
829         actions.add(add); actions.add(edit); actions.add(toggle); actions.add(clockIn); actions.add(clockOut);
830         add(actions, BorderLayout.NORTH);
831
832         add.addActionListener(e -> onAdd());
833         edit.addActionListener(e -> onEdit());
834         toggle.addActionListener(e -> onToggleActive());
835         clockIn.addActionListener(e -> onClockIn());
836         clockOut.addActionListener(e -> onClockOut());
837
838         reload();
839     }
840 }
841
842 // ---- Main Method ----
843 // HayLee - Main Method
844 public static void main(String[] args) {
845     JFrame frame = new JFrame("FarmStoreManager");
846     frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
847     frame.add(new StorePanel());
848     frame.add(new EmployeesPanel());
849     frame.add(new TimeClockPanel());
850     frame.pack();
851     frame.setVisible(true);
852 }
853
854 // ===== End of File =====
```

```
File Edit Selection View ... Search
J FarmStoreManager.java 9+ x
C:\Users\Super\Downloads\Adv.Java\M4\J FarmStoreManager.java > FarmStoreManager > EmployeesPanel > EmployeesPanel()
807
808 reload();
809 }
810
811 void reload() {
812     model.setRowCount(rowCount:0);
813     List<Employee> emps = EmployeeRepo.all();
814     for (Employee emp : emps) {
815         boolean open = TimeRepo.openShiftFor(emp.id).isPresent();
816         model.addRow(new Object[]{emp.id, emp.name, money(emp.hourlyRate), yn(emp.active), yn(open)});
817     }
818 }
819
820 Optional<Employee> selected() {
821     int r = table.getSelectedRow(); if (r < 0) return Optional.empty();
822     String empId = (String) model.getValueAt(r, column:0);
823     return EmployeeRepo.byId(empId);
824 }
825
826 void onAdd() {
827     String name = JOptionPane.showInputDialog(this, message:"Employee Name:"); if (name == null || name.isBlank()) return;
828     double rate = d(JOptionPane.showInputDialog(this, message:"Hourly Rate:"));
829     List<Employee> all = EmployeeRepo.all();
830     all.add(new Employee(id:null, name, rate, active:true));
831     EmployeeRepo.saveAll(all);
832     reload();
833 }
834
835 void onEdit() {
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS

va-project\bin\ 'FarmStoreManager'
PS C:\Users\Super>

Ln 807, Col 1 Spaces: 4 UTF-8 CRLF () Java Go Live

```
File Edit Selection View ... Search
J FarmStoreManager.java 9+ x
C:\Users\Super\Downloads\Adv.Java\M4\J FarmStoreManager.java > FarmStoreManager > EmployeesPanel
835
836 void onEdit() {
837     Optional<Employee> opt = selected(); if (opt.isEmpty()) {
838         JOptionPane.showMessageDialog(this, message:"Select an employee");
839         return;
840     }
841     Employee e = opt.get();
842     String name = JOptionPane.showInputDialog(this, message:" Name:", e.name); if (name == null) return;
843     double rate = d(JOptionPane.showInputDialog(this, message:"Hourly Rate:", Double.toString(e.hourlyRate)));
844     List<Employee> all = EmployeeRepo.all();
845     for (Employee x : all) if (x.id.equals(e.id)) { x.name = name; x.hourlyRate = rate; }
846     EmployeeRepo.saveAll(all);
847     reload();
848 }
849
850 void onToggleActive() {
851     Optional<Employee> opt = selected(); if (opt.isEmpty()) {
852         JOptionPane.showMessageDialog(this, message:"Select an employee");
853         return;
854     }
855     Employee e = opt.get();
856     List<Employee> all = EmployeeRepo.all();
857     for (Employee x : all) if (x.id.equals(e.id)) x.active = !x.active;
858     EmployeeRepo.saveAll(all);
859     reload();
860 }
861
862 void onClockIn() {
863     Optional<Employee> opt = selected(); if (opt.isEmpty()) {
```

PROBLEMS 17 OUTPUT DEBUG CONSOLE TERMINAL PORTS

va-project\bin\ 'FarmStoreManager'
PS C:\Users\Super>

Ln 834, Col 1 Spaces: 4 UTF-8 CRLF () Java Go Live


```
C:\Users\Superx\Downloads> Adv.Java> M4> J FarmStoreManager.java> FarmStoreManager> ReportsPanel> reload()

928     List<Sale> sales = SaleRepo.all();
929     double total = sales.stream().mapToDouble(s->s.total).sum();
930     model.addRow(new Object[]{"Lifetime Sales", money(total)});
931     model.addRow(new Object[]{"Receipts", sales.size()});
932
933     // HayLee - Employee hours and pay report
934     double empHours = EmployeeRepo.all().stream().mapToDouble(e->TimeRepo.sumHours(e.id)).sum();
935     double empPay = EmployeeRepo.all().stream().mapToDouble(e->TimeRepo.sumPay(e.id)).sum();
936     model.addRow(new Object[]{"Total Employee Hours", String.format(format:"%.2f", empHours)});
937     model.addRow(new Object[]{"Total Employee Pay", money(empPay)});
938 }
939 }
940 }
941 }
```

va-project\bin' 'FarmStoreManager'

PS C:\Users\Superx